Week 10 — Linux Processes and Signals

Format: seven hands-on-first lessons using the canonical `projects/linux-system-programming` lab

Suggested workload: approximately 21 focused hours, adjustable to mastery

> Predict the observable state, run one bounded experiment, explain the ownership and lifetime, and stop the claim where the evidence stops.

The project is an offline C17/POSIX lab. It creates only descendants of the invoking process, sends signals only to itself or those descendants, reaps every child, and needs neither root nor network access. The wrappers in `challenges/` do not patch or copy the project. Each follows:

```
purpose → prediction → evidence → explanation → limits
```

Run wrappers from this week directory. Each wrapper resolves the canonical project relative to itself, builds only through the project's existing Makefile, uses checked commands, and labels optional-tool skips.

Daily roadmap

1. Process mental model — `task_struct`, PID identity, address space, files, credentials, kernel state, and process versus program.
2. Fork, copy-on-write, and lineage — duplicated execution context, shared open-file descriptions, private memory illusion, and parent/child state.
3. Exec, ELF, argv, env, and descriptors — image replacement, initial stack, environment, and `FD_CLOEXEC`.
4. Wait, zombies, orphans, and reparenting — status decoding, retained exit records, deterministic subreaping, and lifetime cleanup.
5. Signals and safe return — `sigaction`, masks, pending state, async-signal-safety, self-pipe, and architecture-limited signal-frame discussion.
6. Shell execution and job-control concepts — fork/exec/wait orchestration, process groups, sessions, terminal foreground ownership, and a safe bounded observation lab.
7. Integrated debugging report — `/proc`, `strace`, GDB, injected failures, and a version-disciplined ToyOS/xv6/Linux source comparison.

Each day also has one executable wrapper and one generated four-question HTML theory check. `quizzes.json` is the source of quiz content.

Safety and evidence discipline

Do not paste bare commands from the prose. Use the wrapper so paths, build preconditions, time bounds, expected markers, and cleanup behavior remain visible. PID values, scheduling, syscall spellings, and `/proc` snapshots vary. The stable evidence is the checked invariant: decoded status, observed state, expected `errno`, a marker emitted by the canonical program, or a documented optional-tool skip.

The orphan exercise uses Linux `PR_SET_CHILD_SUBREAPER`; this intentionally avoids depending on container PID 1. The zombie is observed briefly and then reaped. The signal exercises target the current process or a child. The shell day observes the current non-interactive environment and creates a bounded child group; it does not implement an interactive shell or manipulate an unrelated terminal.

Linux source landmarks and version discipline

Useful current-tree landmarks include `include/linux/sched.h`, `kernel/fork.c`, `kernel/exit.c`, `kernel/signal.c`, `fs/exec.c`, `fs/binfmt_elf.c`, `fs/proc/`, and architecture-specific syscall/signal code under `arch/<isa>/`. Names and call paths change. Record the kernel release (`uname -r`), source revision or tag, configuration, architecture, and exact file before making line-level claims. A source landmark explains an implementation; the man-page/POSIX contract defines what an application may rely on.

Architecture comparison is deliberately narrow. Syscall entry and signal frame construction/return are architecture-specific. Fork/exec/wait policy, PID namespaces, file-table lifetime, credentials, and process-group rules are kernel and API concepts, not properties to infer from x86-64 register names.

Completion checkpoint

Produce a report that can answer:

1. What state is copied, shared, reset, or replaced across `fork` and `exec`?
2. Which object owns a descriptor's offset, and when does close-on-exec act?
3. Why is a zombie not a running process, and who must consume its status?
4. When blocked, pending, selected, delivered, handled, and returned, where can a signal be observed?
5. How do session, process-group, and terminal foreground identities differ?
6. Which conclusion came from source, ELF metadata, `/proc`, syscall trace, debugger state, or a checked program assertion?
7. What remains unproved because of timing, namespace, tool, kernel-version, or architecture limits?

Do not auto-fill mastery, confidence, hours, or completion.

Day 1 — A Process Is More Than a Program

Target time: approximately 3 hours

> An executable file is passive input. A process is a live kernel-managed execution context with identity, resources, authority, and changing state.

1. Start with a bounded observation

Run `challenges/day-001-process-model.sh` from the Week 10 directory. Before running it, predict which facts remain constant across `/proc/self/status`, `/proc/self/stat`, and `/proc/self/exe`, and which can differ on every run. The wrapper builds the canonical project, runs `proc_lab`, identifies its ELF, and prints selected source and host facts. It does not inspect arbitrary processes.

Expected stable outcomes are `proc_inspection=ok`, an ELF executable whose machine matches the host build, and status/stat records referring to the process that opened them. Exact PIDs, parent PIDs, scheduling fields, virtual addresses, and executable paths can vary. Explain invariants rather than memorizing a sample.

2. Process versus program

A program is usually a file containing instructions and metadata. An ELF file can describe loadable segments, an entry point, interpreter requirements, symbols, and relocation information. It has no current instruction pointer, open descriptor table, pending signals, scheduler state, or effective credentials merely because it exists.

A process is one execution context known to the kernel. Linux represents the schedulable entity with `struct task_struct`; threads are tasks that share selected resources. The familiar “process” is therefore not one indivisible C object. It is a graph of referenced kernel objects: memory descriptors, file tables, filesystem context, signal-handling state, credentials, namespaces, and scheduler/accounting state. Some references may be shared with sibling threads or related tasks. The exact fields are internal and version-dependent.

Starting the same executable twice creates distinct process identities and mutable states. Conversely, `execve` can replace one process's program image without changing its PID. That single fact defeats the shorthand “a process is a running program.” The phrase is useful only as an introduction.

3. Build an ownership ledger

Use this conceptual ledger, then verify only the fields exposed by the lab:

```
state                          conceptual owner/reference
PID and task state             kernel task identity
virtual mappings               memory descriptor/address space
descriptor numbers             per-process file descriptor table
open offset/status flags       shared open-file description
cwd/root/umask                 filesystem context
UID/GID/capabilities           credential object
blocked signal mask            calling thread
signal dispositions            process-wide signal state
pending signals                thread- or process-directed queues
registers/kernel stack         individual task
```

This distinction predicts behavior. After a normal `fork`, parent and child have separate descriptor tables whose entries refer to the same open-file descriptions, so an offset may be shared while closing one descriptor table entry does not close the other's entry. Threads created with suitable `clone` flags can share the table itself. Never infer sharing merely because two integer descriptor values are equal.

Credentials are similarly richer than one UID. Real, effective, and saved IDs, supplementary groups, capabilities, securebits, namespaces, and no-new-privileges can affect authorization. `/proc/PID/status` exposes a projection, not every policy decision.

4. PID identity and namespaces

A PID is a number in a PID namespace, not a globally eternal handle. It can be reused after a task is fully released. Recording only `1234` is therefore weak evidence. Add the namespace context, observation time, parent relationship, executable identity, and, where relevant, a pidfd. A process inside a container may see a different PID than an observer in an ancestor namespace.

`getpid()` reports the thread-group ID used as the conventional process ID. Linux also has task IDs for individual threads. `/proc/self` resolves relative to the reading process, while `/proc/thread-self` names the calling thread. Day 1 stays with single-threaded programs so those identities do not blur the experiment.

5. Address space and kernel state

The address space gives user code a virtual view. It includes executable segments, shared libraries, stack, heap, mappings, and unmapped gaps. The page tables and virtual-memory areas are kernel-managed state; physical frames can be shared, copied, reclaimed, or backed by files without changing the program's ordinary pointer values.

Not all process state lives in the address space. Descriptor-table entries, credentials, pending signals, wait relationships, accounting, namespace membership, and scheduler state are kernel objects. A core dump or userspace memory inspection therefore cannot by itself reconstruct every process property.

`/proc/PID/maps` would expose mappings, but the canonical `proc_lab` limits itself to status, stat, and exe. That restraint matters: do not claim the wrapper measured copy-on-write pages or permissions it never read.

6. Files and lifetime

The word “file” hides several identities. A descriptor is a small index in a table. Its entry points to an open-file description, which owns current offset and file status flags. That object refers onward to filesystem objects or a pipe/socket/device implementation. Descriptor flags such as `FD_CLOEXEC` belong to the descriptor entry, not the shared open-file description.

Reference counts let a kernel object outlive one name or one descriptor. Closing a descriptor releases a reference. Process exit closes remaining descriptors. An unlinked regular file can remain accessible through an open description. These are lifetime rules, not ELF or CPU architecture facts.

In C, check every operation at the boundary where failure is reported. `open`, `read`, `readlink`, `fclose`, and `snprintf` have different success contracts. The project's `proc_lab` checks truncation before using its path, leaves room for `readlink` termination, checks stream errors, and closes the stream. Error handling is part of the process model because leaks and stale references alter lifetime.

7. Linux implementation landmarks

For a chosen Linux revision, begin with:

- `include/linux/sched.h` for `task_struct` and related declarations;
- `kernel/fork.c` for task creation and shared/copied resources;
- `fs/file.c` and `include/linux/fdtable.h` for descriptor tables;
- `mm/` and `include/linux/mm_types.h` for memory state;
- `kernel/cred.c` and `include/linux/cred.h` for credentials;
- `fs/proc/` for the exported observations used today.

Do not quote current mainline line numbers against the running distribution kernel. Record `uname -r`; obtain the matching source package/config if making an implementation claim. Field names are not an application ABI.

8. Architecture lens: only where it belongs

The ELF `Machine` field and syscall entry path are architecture-specific. x86-64 commonly enters via `syscall`; AArch64 uses `svc`; RISC-V uses `ecall`. Register assignments and entry assembly differ. Once the request reaches generic process-management code, the conceptual PID, file, credential, and wait policies are not explained by the entry instruction.

The wrapper identifies ELF metadata but does not disassemble the entry gate or prove which instruction libc used. State that limit explicitly.

9. Explain the evidence

Write a one-page process inventory. Separate:

1. identity: PID, PPID, executable link, ELF hash;
2. user memory: what an address space conceptually contains;
3. kernel references: files, credentials, namespaces, signal and wait state;
4. mutable task state: scheduling state, registers, accounting;

5. evidence type: program assertion, procfs text, ELF metadata, or source;
6. variation: values expected to differ next run.

Then answer the central question: replacing an image with `execve` preserves process identity while changing program content, and running one ELF twice creates two identities. A program and a process cannot be synonyms.

Practical challenge

Purpose: inventory the identities, owners, and lifetimes that make a Linux process more than executable bytes.

Prediction: the canonical observer reports its PID/PPID, stat record, executable target, and `proc_inspection=ok`; exact IDs vary.

Evidence: checked C output, selected `/proc` fields, project source, ELF header, artifact hash, and recorded kernel release.

Limits: no arbitrary process inspection, namespace traversal, memory-map measurement, thread sharing, privilege transition, or syscall-entry trace is performed.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Process Model

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 10-linux-processes-and-signals/challenges/day-001-process-model.sh
```

- View lab source on GitHub

Day 2 — Fork, Copy-on-Write, and Parent–Child State

Target time: approximately 3 hours

> `fork` creates a new task and a new return path. “Copies the process” is a useful prediction only after every resource is classified as copied, shared, reference-counted, or reset.

1. Run the lineage experiment first

Run `challenges/day-002-fork-cow.sh`. Predict that the canonical `tree` mode will report two distinct child PIDs, each naming the wrapper program's child as parent, and that the complete Week 10 test will still pass. Predict also that the source will show a pipe created before either fork.

The pipe is deliberate. Output ordering from multiple processes is unstable, so each child writes one fixed-size record to a kernel pipe and exits. The parent reads two records and waits until no child remains. The invariant is the relationships in each record, not which child reports first.

2. One call, two returns

On success, `fork()` returns once in each resulting process. The parent receives the child's positive PID; the child receives zero. On failure there is no child and the caller receives `-1` with `errno` set. Correct C code tests failure before branching on zero.

Both tasks resume from the logical point after `fork`, but they are not executing one shared C stack. Each has its own virtual address space view and register state. Local variables initially contain the same bytes, including copies of pipe descriptor integers, yet subsequent ordinary writes in one address space are private unless the memory was explicitly shared.

The scheduler chooses which task runs first. Parent-first output is not a contract. A successful run that happened to print children in index order does not prove ordering.

3. Classify state precisely

A useful fork ledger is:

```
property                          relationship after fork
PID/task identity                 new and distinct
PPID relationship                child points to parent
user virtual mappings             logically copied, usually COW
page frames                       initially shareable until writes
descriptor table entries          copied entries
open-file descriptions            referenced/shared
file offset/status flags           shared through open description
descriptor FD_CLOEXEC bit          copied per descriptor
signal dispositions               inherited
blocked signal mask               inherited
pending signals                   child starts with none
credentials/cwd/root/umask         inherited via kernel rules
CPU usage and selected timers      reset or separately accounted
fork return value                  deliberately different
```

Consult `fork(2)` for the complete application contract. Linux has extensions and corner cases, and multithreaded fork adds severe restrictions before exec. A compact table should never masquerade as the full specification.

4. Copy-on-write is an implementation strategy

Eagerly copying every writable page would be expensive, especially when the child immediately calls `execve`. Linux normally maps parent and child to the same physical pages read-only or with equivalent write-protection accounting. When either writes, a fault lets the kernel create a private writable copy. This is copy-on-write (COW).

COW means “same initial contents, private write semantics,” not “one shared C object.” It can be defeated or altered by explicitly shared mappings. Kernel page deduplication, huge pages, pinning, and memory pressure complicate the physical story. The API guarantee is the process-visible memory behavior; the exact frame choreography is implementation evidence.

The canonical lab does not inspect page-frame numbers or minor-fault counters. Its `tree` mode demonstrates lineage and inherited pipe access, not COW page allocation. The wrapper prints source landmarks and labels COW as a reasoned implementation comparison.

5. Why the pre-fork pipe works

Before forking, the parent creates one pipe with read and write descriptors. Each child inherits entries referring to the same pipe endpoints. It closes the read endpoint, writes a complete record with a checked loop, closes its write endpoint, and calls `_exit`.

The parent closes its write endpoint before reading. This is vital lifetime discipline: a read waiting for EOF would never see EOF while any write reference remains open. After two exact-size reads, it closes the read end and waits until `wait` reports `ECHILD`.

`checked_write` retries `EINTR` and handles partial writes. `checked_read` retries interruption, rejects premature EOF, and advances by the number of bytes actually transferred. Even though a small pipe record is normally atomic relative to `PIPE_BUF`, robust code still checks the syscall result.

The child uses `_exit`, not `exit`. After fork, copied stdio buffers may contain bytes buffered before the split; `exit` could flush them again. In larger programs, inherited `atexit` handlers and libc state are another reason the pre-exec child path should stay narrow.

6. Parent and child are relationships, not ownership

The parent does not “own” all child memory or files. The kernel records relationships needed for signals, wait, and reparenting. The child has an independent PID and can outlive the original parent. Permissions for inspection or signaling depend on credentials, namespaces, ptrace policy, and other checks, not simply ancestry.

After a child terminates, selected identity and status remain so an eligible waiter can consume them. Day 4 names that state a zombie. Parent/child relationships therefore have a post-execution lifetime as well as a creation path.

7. Failure and cleanup paths

Fork can fail because of process limits, memory pressure, cgroup limits, or kernel constraints. In a loop, failure after one successful child creates a partial-construction problem: the parent must close endpoints and reap already-created children. The compact lab exits on fork failure and is aimed at a controlled environment; production code should preserve enough state to clean up partial success.

Every descriptor should have a clear owner in every branch. Every child should have a known reaper. Avoid C patterns that duplicate cleanup across many branches; structured labels can be safer if they preserve the first failure and never close an uninitialized descriptor.

8. Linux source landmarks

For a recorded source revision inspect `kernel/fork.c`, especially the creation path around `kernel_clone`/`copy_process` as named in that revision. Memory duplication work is under `mm/`, while descriptor-table operations are under `fs/` and signal structures under `kernel/signal.c`. Look for the flags that determine whether a resource is shared or duplicated rather than assuming every task creation is ordinary `fork`.

Linux libc may implement `fork` using a clone-family syscall. `strace` may therefore display `clone` or `clone3` rather than a literal `fork`. That is not a contradiction; it is a distinction between the C API and kernel entry used by that libc/kernel combination.

9. ToyOS and xv6 comparison

ToyOS in this course has a deliberately smaller process story; verify its current milestone before claiming fork or COW support. If absent, compare an absence, not imagined code.

xv6-riscv's `kernel/proc.c` contains educational process allocation, fork, exit, and wait paths. Its page-table copying policy depends on the exact revision; classic teaching versions eagerly copy user pages, while COW labs change that behavior and add reference counting and fault handling. Record the repository commit and whether the COW lab is applied.

Linux supports richer sharing through clone flags, threads, namespaces, pidfds, cgroups, overcommit policy, and many memory-mapping cases. More features mean more lifetime states, not a different basic need to classify each resource.

10. Write the causal account

Explain the canonical run without saying merely “fork copied the parent”:

1. the root created a pipe;
2. each successful fork created a distinct PID and return value;
3. inherited descriptor entries referenced the pipe;
4. each child sampled PID/PPID into a record and used checked `write`;
5. children closed references and `_exit`ed;
6. the parent consumed two records, closed its endpoint, and reaped all;
7. record order remained unconstrained.

Then separate observed evidence from expected COW implementation. If you want to claim actual faults or page sharing, design another bounded measurement with matching-kernel facilities; today's output is insufficient.

Practical challenge

Purpose: classify parent/child identities and inherited resources without turning “copy” into an inaccurate blanket statement.

Prediction: two unique child PIDs report the same parent PID; records may arrive in either order; all children are reaped.

Evidence: canonical source, checked `tree` output, process-related trace when `strace` is present, project test marker, and artifact identity.

Limits: the exercise does not measure physical pages, prove write-fault COW, cover multithreaded fork, inject partial-loop fork failure, or establish scheduler ordering.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Fork Cow

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 10-linux-processes-and-signals/challenges/day-002-fork-cow.sh
```

- View lab source on GitHub

Day 3 — Exec, ELF, argv, env, and Descriptor Inheritance

Target time: approximately 3 hours

> A successful `execve` does not create a new process and does not return. It replaces the calling process image while selected kernel state survives.

1. Begin with the status mode

Run `challenges/day-003-exec-elf-inheritance.sh`. Predict a child that executes `/bin/sh -c 'exit 7'`, a parent that decodes exit 7, and an ELF inspection of the canonical lab. The same mode creates another child, terminates it with `SIGTERM`, and distinguishes signal termination from a normal numeric exit.

The wrapper supplies commands, expected markers, paths, and limits. Do not substitute an unbounded trace of a login shell. The canonical child executes a fixed command and the parent reaps it.

2. The exec contract

`execve(path, argv, envp)` asks the kernel to replace the caller's executable image. On success, old user mappings are discarded according to exec rules, new mappings and an initial stack are created, execution begins at an entry selected through the executable format and dynamic-loader contract, and the call never returns. The PID remains the same.

On failure, `execve` returns `-1`, sets `errno`, and the old image continues. That asymmetry requires immediate error handling. In a post-fork child, report the error through a safe channel if needed and call `_exit` with a documented mapping. The project maps `ENOENT` to 127 in its injected-failure mode, a shell convention rather than a kernel-mandated universal mapping.

The convenience function `execl` used by the lab constructs `argv` and inherits the caller's environment. Its variadic list must end with a correctly typed null pointer. The kernel-facing concept remains exec.

3. ELF is a format contract, not the whole launch

For a native ELF executable, the header names class, byte order, machine, type, and entry. Program headers describe segments and attributes. A dynamically linked executable commonly names an interpreter in `PT_INTERP`; the kernel loads the interpreter and transfers control according to the ABI. The dynamic linker maps dependencies, performs relocations, initializes runtime state, and eventually reaches application startup.

`main` is not normally the ELF entry point. Startup code receives the initial stack, prepares libc, and invokes `main`. A stripped binary can run without a symbol named `main`; symbols aid linking/debugging but are not the execution contract.

Scripts add another path: a shebang can select an interpreter. Exact parsing and recursion limits are OS implementation details. Today's child explicitly execs `/bin/sh`; it does not rely on a script shebang.

4. `argv`, environment, and the initial stack

`argv` is a null-terminated array of pointers to argument strings, conventionally with `argv[0]` naming the invoked program. The kernel does not enforce every social convention about that string. `envp` is another null-terminated pointer array whose strings commonly have `NAME=value` form. Together with auxiliary vector entries and architecture ABI data, these appear in the new image's initial user stack.

Argument and environment bytes are bounded by system limits and available space. Treat environment input as untrusted: variables can alter locale, allocation, dynamic loading, tool behavior, and child configuration. Privileged execution applies additional rules, but this lab performs no privilege transition.

The old C pointers do not survive as meaningful objects. The kernel copies launch data into the new image. A successful exec destroys the old userspace heap and stack; only values intentionally represented in surviving kernel state, files, arguments, environment, or external storage can cross.

5. State that survives, resets, or closes

A disciplined exec ledger includes:

```
PID and parent relationship       preserved
new executable mappings/stack     replaced
ordinary caught dispositions      reset to default
ignored signal dispositions       generally remain ignored
blocked signal mask               preserved
pending signals                   preserved by Linux rules
open descriptors                  preserved unless FD_CLOEXEC
open-file descriptions/offsets    remain behind surviving descriptors
credentials                       transformed by documented exec rules
cwd/root/umask                    preserved
threads other than caller         do not survive
```

Consult `execve(2)` for the complete list. Memory locks, alternate signal stacks, mappings, timers, shared memory attachments, robust mutex state, and dumpability have specific rules. “Everything stays except code” and “everything resets” are both wrong.

6. Descriptor inheritance and close-on-exec

Descriptors are inherited by default. The `FD_CLOEXEC` descriptor flag tells the kernel to close that entry during successful exec. Set it atomically when creating or accepting a descriptor (`O_CLOEXEC`, `pipe2`, appropriate accept variants) when possible. A separate `fcntl` after creation leaves a race in a multithreaded program: another thread could exec between creation and flagging.

Close-on-exec does not destroy an open-file description if another descriptor still references it. It closes one descriptor-table reference. Offset and file status flags belong to the open description; `FD_CLOEXEC` belongs to the entry.

The canonical status mode inherits ordinary descriptors but does not build a dedicated proof of `FD_CLOEXEC`. The wrapper exposes source and documentation landmarks; it must not claim a runtime close-on-exec observation. A follow-up lab could open two harmless pipes, atomically mark one endpoint, exec a fixed helper, and have the helper enumerate only its own `/proc/self/fd`.

7. Error handling after fork

Between `fork` and `exec` in a multithreaded process, only async-signal-safe operations are generally safe until exec. Locks may have been held by vanished threads in the child copy. Prefer `posix_spawn` where it expresses the launch, or keep the child path to prepared descriptor operations and exec.

The lab is single-threaded. Its failure path checks `errno` immediately and uses `_exit(127)` or `_exit(126)`. It does not call `perror`, allocate, or flush copied stdio state in the child. The parent alone interprets wait status.

Always distinguish “could not execute” from “new program ran and returned a code.” An error pipe with close-on-exec can carry pre-exec `errno`: EOF implies successful exec closed the write end, while a fixed-size error record reports failure. That design is beyond the compact lab but follows directly from lifetime analysis.

8. Linux source landmarks and version discipline

In a selected Linux revision, inspect `fs/exec.c` for generic exec orchestration, `fs/binfmt_elf.c` for ELF loading, `fs/binfmt_script.c` for shebang handling, and the architecture tree for register setup and entry details. Descriptor close-on-exec handling connects to file-table code.

Record the exact kernel tag and configuration. Distribution kernels may carry patches; current mainline names can differ from a running release. ELF details also depend on compiler/linker flags and whether the output is PIE. Hash and inspect the actual artifact rather than describing a generic binary.

9. Architecture lens

ELF machine type, relocation forms, ABI registers, initial stack alignment, and transfer to user mode vary by ISA. Syscall entry also varies. The generic exec policy—replace image, preserve PID, apply descriptor and signal rules—is not usefully explained by comparing x86 `syscall`, AArch64 `svc`, and RISC-V `ecall`. Keep architecture discussion at the entry, ELF, and user-register boundary.

10. Compare ToyOS, xv6, and Linux

ToyOS should be described only from its implemented process milestone. If it loads one fixed user image and lacks exec, say so. Do not retrofit Linux's argument, environment, or descriptor semantics onto it.

xv6-riscv provides a compact exec path, typically in `kernel/exec.c`, that builds a user address space and argument stack for its simpler filesystem and ABI. Verify the revision. Linux adds executable-format registration, dynamic linking contracts, credentials, namespaces, security hooks, close-on-exec, auxiliary vectors, ASLR, and broad compatibility.

11. Explain the checked run

Write the sequence:

1. parent forks and checks failure;
2. child calls `execl` with fixed argument vector;
3. success replaces the child image while retaining its PID;
4. `/bin/sh` exits 7;
5. parent waits for the exact PID;
6. `WIFEXITED` guards `WEXITSTATUS`;
7. a separate child is signaled, so `WIFSIGNALED` guards `WTERMSIG`.

Then classify each claim as source, ELF metadata, runtime marker, or API contract. The run does not reveal the full initial stack or dynamic-linker relocations merely because `readelf` describes the file.

Practical challenge

Purpose: trace image replacement while preserving process identity and classify argument, environment, signal, and descriptor state.

Prediction: the fixed exec child exits 7, the second child is terminated by `SIGTERM`, and wait decoding keeps these outcomes distinct.

Evidence: checked mode output, exact source, ELF headers/program headers, artifact hash, and an optional process trace.

Limits: no privileged exec, multithreaded fork/exec, close-on-exec runtime probe, dynamic-linker relocation audit, shebang recursion, or full initial stack dump is performed.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Exec Elf Inheritance

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 10-linux-processes-and-signals/challenges/day-003-exec-elf-inheritance.sh
```

- View lab source on GitHub

Day 4 — Wait Status, Zombies, Orphans, and Reparenting

Target time: approximately 3 hours

> Exit ends execution, not every trace of process identity. A waiter must consume status before the kernel can release the retained child record.

1. Observe, then always reap

Run `challenges/day-004-wait-zombies-orphans.sh`. Predict three outcomes: normal exit 7 and signal termination remain distinguishable; a child reaches `Z` briefly before its parent reaps it; and an orphaned grandchild is reparented to the lab's subreaper rather than relying on the host's PID 1.

The zombie mode polls only a child it created, for at most 100 one-millisecond delays. Once state `Z` is observed, it immediately calls `waitpid` and checks exit 23. This is a controlled lifetime demonstration, not permission to leave zombies behind.

2. Wait status is encoded

`waitpid` returns a PID on success, zero only with suitable nonblocking options when no selected child has changed state, and `-1` on error. The integer status is encoded. Test the corresponding predicate before using an extractor:

- `WIFEXITED(status)` before `WEXITSTATUS(status)`;
- `WIFSIGNALED(status)` before `WTERMSIG(status)`;
- `WIFSTOPPED(status)` before `WSTOPSIG(status)`;
- `WIFCONTINUED(status)` for a continued child where supported/requested.

Do not compare raw status to an exit code. Do not call `WEXITSTATUS` for a signaled child. `waitid` offers a different interface and `siginfo_t`; choose an API deliberately rather than mixing representations.

`waitpid` can be interrupted. Whether to retry depends on the program's event model. Waiting for one known child should also verify that the returned PID is the requested PID, as the canonical status mode does.

3. Zombie is retained status, not running code

When a child exits, its user execution is over and most resources are released. Linux retains enough task identity, accounting, and status for an eligible parent to wait. That retained state appears as zombie state. It is not consuming CPU and cannot resume ordinary execution.

The parent creates the zombie by delaying wait, not because the child forgot to clean itself. The cure is a correct reaping policy. Signals alone do not “kill a zombie”; execution has already ended.

Polling `/proc/PID/stat` requires care because the command name is enclosed in parentheses and can contain awkward characters. The lab's generated child has a controlled name, so its compact `fscanf` is acceptable only within that scope. General parsers should locate the final closing parenthesis according to the procfs format and handle PID reuse.

4. Orphaning and reparenting

An orphan is a process whose original parent has terminated while it still exists. Linux reparents it to an eligible child subreaper or ultimately the namespace's init process. Reparenting supplies a task that can receive status and reap later; it does not terminate the orphan.

Containers make “PPID becomes 1” an unreliable teaching assertion. The canonical mode calls `prctl(PR_SET_CHILD_SUBREAPER, 1)`, then creates a middle child and grandchild. A readiness pipe ensures the grandchild records the middle PID before the middle exits. The grandchild waits boundedly for PPID change, reports before/after IDs, and exits. The top process waits for both descendants and verifies adoption.

This is Linux-specific. Subreapers are used by supervisors to contain daemonizing descendants, but a production supervisor also needs signal forwarding, shutdown policy, repeated nonblocking waits, and race handling.

5. Lifetime and C discipline

Every successful fork adds a potential wait obligation. Maintain a ledger:

```
middle child       created → exits → status consumed by original parent
grandchild         created → orphaned → adopted → report read → reaped
pipe read end      parent owns until exact report arrives
pipe write end     descendant owns until report or failure
```

Close unused endpoints immediately. Otherwise an EOF-dependent reader can block because it unknowingly owns a remaining writer. Use fixed-size records with checked partial I/O. Preserve `errno` before cleanup if the diagnostic depends on it.

The lab's failure mode waits for one child, then calls `waitpid` again and requires `-1/ECHILD`. This injected failure proves the status was consumed and there is no waitable child. It does not prove behavior for every PID-selection option.

6. Races and better handles

Process exit, `/proc` lookup, signal delivery, and PID reuse can race. A PID read earlier can later name no process or a different process after reuse. Linux pidfds provide stable references for selected operations and integrate with polling. They are an important production tool but absent from this portable C17/POSIX-centered lab.

Avoid “check then act” logic that assumes a `/proc` state remains true. The zombie observation is evidence at one instant; `waitpid` is the authoritative consumption operation. Similarly, `kill(pid, 0)` is a permission/existence probe at an instant, not a durable reservation.

7. Linux, xv6, and ToyOS landmarks

For a pinned Linux source revision, inspect `kernel/exit.c` for exit, notification, reparenting, and wait paths, plus `fs/proc/array.c` for task state formatting. Names and decomposition move between releases. Record `uname -r`, source tag, architecture, and config.

xv6-riscv's `kernel/proc.c` provides a much smaller exit/wait and parent relationship under its process-table locking rules. Verify whether a given revision reparents to `initproc` and how status is copied. ToyOS may have one fixed user process and no general wait tree; state the actual absence.

Linux adds threads, ptrace relationships, namespaces, subreapers, job-control states, richer wait selection, and many concurrency constraints. The shared invariant is that termination status has an owner and a consumption point.

8. Explain the evidence

For the zombie, narrate fork, child `_exit(23)`, retained status, `/proc` observation, `waitpid`, guarded status decode, and release. For reparenting, narrate both pipes, readiness ordering, middle exit, new parent selection, report, both waits, and cleanup.

Keep three labels distinct: **exited** describes execution; **zombie** describes retained waitable state; **orphan** describes a changed parent relationship. A process can become orphaned while running, and an exited child can be a zombie without being an orphan.

Practical challenge

Purpose: decode child transitions and prove that every created descendant has a bounded cleanup path.

Prediction: state `Z` is observed then reaped; the grandchild changes from the middle PID to the subreaper PID; double wait reports `ECHILD`.

Evidence: checked canonical modes, selected `/proc` state, source, process-only trace when available, and complete Week 10 regression.

Limits: no PID-reuse stress, pidfds, ptrace stops, multithreaded waiting, namespace-init behavior, or production supervisor loop is demonstrated.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Wait Zombies Orphans

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 10-linux-processes-and-signals/challenges/day-004-wait-zombies-orphans.sh
```

- View lab source on GitHub

Day 5 — Signals, Masks, Pending State, and Safe Return

Target time: approximately 3 hours

> A signal is asynchronous notification with selection, masking, disposition, delivery, and return rules. A handler is not an ordinary callback.

1. Run both canonical signal modes

Run `challenges/day-005-signals-safe-handlers.sh`. Predict that `SIGUSR1` becomes pending while blocked, the handler has not run at that point, and unblocking permits delivery. Predict that the `SIGUSR2` handler writes one byte to a self-pipe and normal flow prints the result.

The wrapper also runs checked failures: installing a handler for `SIGKILL` must fail with `EINVAL`, and signal-zero probing of an intentionally impossible PID must fail with `ESRCH`. It sends no signal to unrelated processes.

2. Separate the signal stages

A process- or thread-directed signal is generated, may become pending, is selected for an eligible thread, is delivered according to its disposition, and may cause default action, ignoring, or handler execution. A blocked signal is not “disabled.” It generally remains pending until unblocked, consumed by a synchronous interface, or otherwise resolved.

Traditional signals generally coalesce: repeated generation while one instance is pending need not produce matching handler counts. POSIX real-time signals have queuing and ordering rules. Therefore a single `sig_atomic_t` flag means “something happened,” not “exactly N events happened.”

Signal masks are per-thread. Dispositions are process-wide. Pending state can be thread-directed or process-directed. In a multithreaded program, delivery selection is consequently more complex than “the process runs its handler.” Today's lab is single-threaded.

3. Install with `sigaction`

`sigaction` makes disposition, handler mask, and flags explicit. Initialize the entire structure, initialize `sa_mask`, choose flags deliberately, and check the return. The lab uses `SA_RESTART`, but restart behavior is syscall-specific; never assume every interrupted operation restarts.

During a handler, the delivered signal is ordinarily blocked, along with signals named in `sa_mask`, unless flags change that behavior. Nested handlers can invalidate simple state assumptions. `SIGKILL` and `SIGSTOP` cannot be caught, blocked, or ignored.

For race-free waiting, do not test a flag and then call `pause`: a signal can arrive between those operations. Block first, test state, and use `sigsuspend` to atomically replace the mask while sleeping, or use synchronous mechanisms such as `sigwaitinfo` in a designed signal thread.

4. Async-signal-safety

A handler can interrupt code while libc or application locks and invariants are mid-update. Calling `printf`, `malloc`, most stdio, or ordinary lock-taking code can deadlock or corrupt state. POSIX defines a limited set of async-signal-safe functions. Consult the platform documentation; familiarity is not proof.

The simple handler assigns a `volatile sig_atomic_t`. That type supports the narrow communication intended here, but `volatile` does not make arbitrary shared state atomic or establish a general cross-thread memory model.

The self-pipe handler saves `errno`, attempts one `write` of one byte to a nonblocking pipe, and restores `errno`. Normal flow performs `read`, validation, stdio, and cleanup. Nonblocking mode prevents a full pipe from trapping the handler. The sample intentionally ignores a failed handler write, meaning notifications can be lost if the pipe is full. A production design must make that coalescing/loss policy explicit.

5. Masks and pending evidence

The pending mode installs the disposition before blocking, constructs a set with `sigemptyset` and `sigaddset`, then calls `sigprocmask(SIG_BLOCK, ...)`. `raise(SIGUSR1)` generates the signal for the process. `sigpending` and `sigismember` verify pending membership while `handled` remains zero. Unblocking allows delivery, after which the checked flag is one.

This sequence is deterministic enough for the single-threaded self-signal. It does not measure when an externally generated signal would be selected, or how multiple eligible threads would be chosen. Output from before and after unblock is the evidence; a generic statement that “signals are immediate” is false.

6. Signal return and architecture lens

To invoke a userspace handler, the kernel arranges saved user context and a signal frame according to the architecture ABI, changes user registers to enter the handler, and supplies a return path that invokes signal-return machinery. Linux validates/restores state on return. Exact frame layout, register names, trampoline details, and syscall numbers vary across x86-64, AArch64, and RISC-V.

This is one of the week's two justified architecture lenses. Inspect matching Linux source under `arch/<isa>/kernel/signal.c` and entry code only for the running kernel revision. Do not manually call `rt_sigreturn`, forge frames, or hard-code layout in portable C.

The lab proves handler execution through state and pipe evidence. It does not dump the signal frame or establish the precise instruction at which delivery occurred.

7. Interrupted operations and lifetime

Syscalls may fail with `EINTR`, restart transparently, complete partially, or have API-specific rules. Write retry loops only after understanding whether progress occurred. The process lab advances buffers by successful byte counts; the signal lab retries the normal-flow read only on `EINTR`.

Handler-visible objects must remain live for every possible delivery. Before closing the self-pipe, block/remove the signal or otherwise prove no handler can use the descriptors. The single-threaded sample raises synchronously and completes one delivery before closing, then sets globals to `-1`; its narrow control flow supplies that proof. Long-running programs need stronger shutdown sequencing.

8. Source landmarks and comparison

For pinned Linux source inspect `kernel/signal.c`, `include/linux/signal_types.h`, and matching architecture signal/entry files. Application contracts come from `sigaction(2)`, `signal-safety(7)`, and POSIX; internal function names are not ABI.

xv6-riscv's base tree has a far smaller user signal story (often only kill state checked around trap return); extensions vary, so verify the commit. ToyOS may have interrupt/trap machinery without POSIX process signals. Compare implemented mechanisms, not shared vocabulary.

Linux supports queued real-time signals, thread-directed delivery, signalfd, ptrace interaction, job-control signals, alternate stacks, restart blocks, and architecture-specific frames. The self-pipe remains useful because it converts minimal asynchronous notification into ordinary event-loop work.

9. Build the causal trace

For `pending`, label generation, blocked mask, pending observation, unblock, selection, handler assignment, and return. For `self-pipe`, label installed disposition, nonblocking endpoint, generated signal, safe write, restored `errno`, normal-flow read, and close.

Then list non-claims: no delivery count, no cross-thread ordering, no full pipe stress, no frame capture, no real-time queue, and no proof that `SA_RESTART` restarts all calls.

Practical challenge

Purpose: observe blocked pending state and move handler notification into normal control flow without unsafe library work.

Prediction: `SIGUSR1` is pending before unblock and handled afterward; `SIGUSR2` contributes one byte through the self-pipe.

Evidence: checked output, exact handler source, optional signal trace, expected `EINVAL`/`ESRCH` failures, kernel release, and artifact hash.

Limits: single-threaded, one notification, no saturation, no real-time signals, no alternate stack, and no captured architecture signal frame.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Signals Safe Handlers

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 10-linux-processes-and-signals/challenges/day-005-signals-safe-handlers.sh
```

- View lab source on GitHub

Day 6 — Shell Execution, Process Groups, Sessions, and Job Control

Target time: approximately 3 hours

> A shell is a process orchestrator. Job control adds process-group, session, and controlling-terminal policy; it is not merely fork plus exec.

1. Use the safe bounded lab

Run `challenges/day-006-shell-job-control.sh`. Predict that the wrapper can report its own PID, parent, process group, session, and whether standard input is a terminal. It then runs the canonical fixed fork/exec/wait status mode and uses a short-lived Python helper to create and verify one child process group. All descendants are waited for.

The experiment does not call `setsid`, seize a controlling terminal, use `tcsetpgrp`, stop an interactive job, or implement a shell. Those operations can disrupt the IDE terminal. The lesson builds the model while keeping the runtime observation non-interactive and bounded.

2. A minimal command launch

For a simple external command, a shell conceptually parses input, performs expansion, prepares redirections, forks or uses a spawn facility, adjusts the child's descriptors and process-group membership, execs the target, then waits or records a background job.

Each verb has failure and lifetime consequences. A pipe must exist before the children that inherit it. Redirection opens can fail before exec. Unused pipe ends must close in every process. Exec failure must be distinguished from the new command's exit. The parent must retain enough identity to wait and report status.

Builtins complicate the model. `cd` must affect the shell's own cwd, so executing it only in a child would not implement normal shell semantics. Pipelines, compound commands, substitutions, and traps add further rules. No full shell is required this week.

3. Process groups

A process group is a set of processes identified by a process-group ID (PGID). It lets a shell and terminal direct a signal at a job containing a pipeline rather than one PID. Group membership is not the same as ancestry: siblings and descendants can be arranged into one job group under documented constraints.

`setpgid` is race-sensitive because both shell and child run concurrently. Robust shells arrange calls so the desired grouping succeeds regardless of which side runs first, while handling expected race outcomes carefully. Today's helper creates one child, has the child call `setpgid(0,0)`, reports its IDs over a pipe, and has the parent verify/reap it. That demonstrates a group leader whose PID equals PGID; it does not claim pipeline-grade race handling.

Signals can target a process group using APIs with the documented negative PID convention or terminal-generated delivery. Never experiment with broad negative targets in an uncontrolled environment. The wrapper sends no job-control signals.

4. Sessions and controlling terminals

A session contains one or more process groups. A session leader can acquire a controlling terminal under system rules. The terminal tracks one foreground process group. This gives the kernel terminal driver a destination for interactive signals such as `SIGINT`, `SIGTSTP`, and `SIGQUIT`.

Background groups that attempt terminal reads can receive `SIGTTIN`; terminal-mode-dependent background writes can receive `SIGTTOU`. When the foreground job stops or exits, an interactive shell regains terminal foreground ownership and restores terminal modes.

PID, PPID, PGID, SID, and foreground PGID answer different questions:

```
PID   which task/thread-group identity?
PPID  which current parent relationship?
PGID  which job-oriented process group?
SID   which session?
tpgid which group currently owns terminal foreground access?
```

A non-interactive IDE task may have no controlling terminal on stdin. That is a valid result, not a failed lab. `/proc/self/stat` exposes identifiers, while `isatty` and terminal APIs establish whether a descriptor is a terminal.

5. Pipelines and descriptor lifetime

For `producer | consumer`, the shell creates a pipe, starts both commands in one job group, maps producer stdout to the write end and consumer stdin to the read end, and closes all unrelated copies. If the shell retains a write end, the consumer may wait forever for EOF. If a child inherits unrelated descriptors across exec, resources and privileges leak.

Use atomically close-on-exec descriptors where available. After `dup2`, close the original endpoints. Treat every launch as a transaction with partial success: if the second fork fails, terminate/reap or otherwise manage the first child and release descriptors.

Exit reporting for pipelines is shell policy. The last command, `pipefail`, signal termination, asynchronous notifications, and stopped/continued states are not encoded by one universal number. Keep kernel wait facts separate from shell presentation.

6. Foreground waiting without races

An interactive shell should not block blindly on one PID when a job contains multiple processes and can stop or continue. It tracks members, waits on the group with suitable options, updates job state, and handles `SIGCHLD` without unsafe handler work or lost wakeups.

A self-pipe or synchronous signal-wait design can integrate child changes into an event loop. The handler should not mutate complex job tables. Signal masks must close the race between checking job state and sleeping. This connects Day 5's notification discipline to shell orchestration.

The canonical lab checks only simple exit and signal termination. It neither handles `WUNTRACED`/`WCONTINUED` nor maintains a job table, so do not claim those paths were tested.

7. Failure boundaries

Name the earliest detector:

- parse/expansion error: shell before process creation;
- pipe/open failure: launcher setup;
- fork/spawn failure: launcher, possibly after partial construction;
- process-group race: `setpgid` caller;
- exec failure: pre-exec child, communicated to parent;
- program exit/signal: wait status;
- terminal ownership error: `tcsetpgrp` and session/terminal rules.

Never collapse all failures to “command not found.” Preserve `errno` and context, but ensure child diagnostics cannot deadlock on inherited stdio locks. The project's fixed ENOENT mapping is one contained example.

8. Linux source and comparison

Pinned Linux landmarks include `kernel/fork.c`, `kernel/exit.c`, `kernel/signal.c`, and tty code under `drivers/tty/`, with UAPI definitions in headers. User-space shell semantics belong to the shell's source and POSIX, not solely the kernel. Record shell version as well as kernel version when investigating interactive behavior.

xv6-riscv's shell is intentionally small and lacks Unix job control; examine `user/sh.c` in the chosen revision. Its fork/exec/pipe/wait structure is excellent for dataflow but not evidence of sessions or terminal foreground groups. ToyOS's fixed user launch is smaller still.

Linux provides the process/session/tty mechanisms, while Bash, dash, zsh, and other shells choose parsing and user-facing policy. Do not attribute shell syntax to a CPU architecture or kernel syscall entry path.

9. Write the bounded report

Record the observed PID/PPID/PGID/SID and terminal result. Explain why the current environment may already place the wrapper in a group created by Cursor's launcher. Then trace the canonical status child through fork, fixed exec, wait, and decode. Finally explain the helper's child-led process group and guaranteed reap.

List what a real interactive shell still needs: parser and expansion rules, redirections, pipelines, race-safe group setup, controlling-terminal handoff, termios save/restore, job table, `SIGCHLD` integration, builtins, and shutdown cleanup.

Practical challenge

Purpose: connect simple fork/exec/wait launch to the identities required for job control without modifying the active terminal.

Prediction: the fixed command exits 7; one helper child becomes a process group leader; terminal availability depends on the invoking environment.

Evidence: checked canonical output, self-only proc/session observations, one bounded child-group report, source, and full descendant cleanup.

Limits: no full shell, parser, pipeline, session creation, terminal foreground transfer, stopping/continuing, background I/O, or signal broadcast is performed.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Shell Job Control

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 10-linux-processes-and-signals/challenges/day-006-shell-job-control.sh
```

- View lab source on GitHub

Day 7 — `/proc`, `strace`, GDB, Failures, and Comparison Report

Target time: approximately 3 hours

> A debugging report is a chain from artifact identity to observation to causal claim. Tool output without version, scope, and limits is not proof.

1. Generate the evidence bundle

Run `challenges/day-007-debug-report.sh`. Predict `WEEK10 PASS`, hashes for the three canonical binaries, checked injected failures, `/proc` output, and either captured or explicitly skipped `strace` and GDB evidence. The wrapper creates its report under a fresh temporary directory and prints the path. It does not modify the project beyond the project's normal build products.

Optional tools are genuinely optional. `SKIP: strace unavailable` is more accurate than fabricated syscall evidence. A debugger blocked by ptrace policy should be reported as unavailable with its diagnostic, not silently converted to success.

2. Establish identity first

Record:

- UTC capture time and `uname -a`;
- compiler, libc/tool, `strace`, and GDB versions where present;
- absolute canonical project path;
- SHA-256 hashes and ELF class/machine/type for each binary;
- exact invocation and expected marker;
- whether output was source, procfs, trace, debugger, or program assertion.

A source tree can differ from the built binary. A binary can differ from the one attached in GDB. A process can exec between two observations. Identity is not paperwork; it is the premise behind symbolic interpretation.

3. Read `/proc` as a changing interface

`/proc` is a kernel-generated view, not stored process memory. Reads at different times need not form one atomic snapshot. Fields are documented by procfs manuals and kernel source, but visibility can be restricted by mount options, namespaces, credentials, LSM policy, or process exit.

The canonical `proc_lab` reads its own `status`, first `stat` line, and `exe` symlink. It checks opens, input, close, `snprintf` bounds, `readlink`, and NUL termination. `/proc/self/exe` can include a deleted marker or be inaccessible under some conditions; today's successful run establishes only this environment.

For the zombie experiment, `/proc/PID/stat` state `Z` is a momentary observation followed by authoritative `waitpid`. For process groups and sessions, parse documented fields carefully; a comm field with spaces makes naive whitespace splitting unsafe in general.

4. Use `strace` for syscall evidence

`strace -f` follows descendants and can show clone-family creation, exec, wait, signal, pipe, read, and write calls. Its display is decoded tool output, not raw kernel source. Syscall names can differ by architecture, libc choice, and tracer decoding. Attaching also changes timing.

Filter narrowly. The wrapper traces only the fixed `process_lab status` mode, stores output in the temporary report directory, and extracts process/signal landmarks. It never traces unrelated processes. Explain an observed `clone`/`clone3` as the kernel interface underlying libc `fork`, if that is what the actual trace shows; do not predict a spelling and call it observed.

Trace return values locate boundaries. An exec line followed by exit 7 differs from an exec returning `ENOENT`. A wait returning a status must still be decoded according to API macros. Tracing does not reveal all userspace state or kernel-internal branches.

5. Use GDB for stopped-state evidence

The project provides `gdb/week10.gdb`, which breaks at `main`, runs, and continues. In batch mode against `proc_lab`, a successful capture combines a known binary, symbol breakpoint, and checked program marker. If debug information is absent, function symbols may still permit a breakpoint, but locals and source lines can be limited.

Before interactive debugging, predict the stop and select variables to inspect. Afterward, label only values actually captured. Fork debugging requires an explicit follow-fork policy; exec changes mappings and symbols; signals can be passed, stopped, or suppressed by debugger configuration. GDB changes process control and timing.

Never confuse a breakpoint at `main` with the process's first instruction. ELF startup and dynamic-loader work precede it. Never present the command file as evidence if GDB did not run.

6. Interpret injected failures

The process failure mode forks and attempts a fixed nonexistent path. In the child, `ENOENT` maps to `_exit(127)`; another exec error maps to 126. The parent checks normal exit and exact 127, then attempts a second wait and requires `ECHILD`.

The signal failure mode asks `sigaction` to install a `SIGKILL` handler and requires `EINVAL`. It then calls `kill(999999999, 0)` and requires `ESRCH`. Signal zero performs permission/existence checks without delivery. The chosen PID is outside ordinary Linux PID ranges in this environment assumption; if a future kernel permits it, the lab's expectation would need revision.

Injected failures are useful because the expected detector and cleanup are specified. They are not fuzzing and do not cover resource exhaustion, partial fork construction, pipe saturation, permission denial, or races.

7. Linux source landmarks, exactly but cautiously

Anchor a source tour to a tag matching the running kernel where possible:

```
include/linux/sched.h     task representation
kernel/fork.c             task creation/resource sharing
fs/exec.c                 generic exec orchestration
fs/binfmt_elf.c           ELF binary format
kernel/exit.c             exit, wait, reparenting
kernel/signal.c           generic signal machinery
fs/proc/                  procfs task presentation
arch/<isa>/entry/         syscall entry
arch/<isa>/kernel/signal.c signal-frame setup/return
```

These are landmarks, not promised stable paths or call graphs. Record commit, config, distribution patches, and architecture. Search within that revision, then cite symbols and lines from it. Separate UAPI guarantees from internals.

Architecture comparison belongs only at syscall entry and signal frame here. x86-64, AArch64, and RISC-V use different trap instructions, register ABIs, entry assembly, and user-context layouts. Generic fork/exec/wait/signal policy continues above those boundaries.

8. ToyOS, xv6, and Linux

Build a comparison by capabilities actually present:

```
question            ToyOS                xv6-riscv             Linux
task model          course milestone     fixed process table    tasks + sharing flags
image launch        fixed/minimal         compact exec           format handlers + ELF
wait/reparent       verify implementation compact exit/wait      namespaces/subreapers
signals             trap-level subset     small kill mechanism   POSIX/Linux signal model
observation         serial/GDB symbols    console/GDB             procfs/ptrace/trace tools
```

Replace broad cells with exact source citations from the revisions used. ToyOS may not implement fork, exec, wait, or POSIX signals; absence is a valid result. xv6 teaching branches vary, especially COW and labs. Linux richness adds concurrency and lifetime obligations rather than making the smaller systems “incorrect.”

Choose one invariant per comparison. For example: who owns exit status; which object keeps an open offset; what state survives image replacement; where a pending asynchronous event is checked; and how a user return context is constructed.

9. Debugging workflow

When output surprises you:

1. identify binary hash, process identity, kernel/tool versions, and command;
2. reproduce with the smallest canonical mode;
3. determine whether failure is before fork, before exec, in the new image, during signal delivery, or during wait;
4. inspect self-owned `/proc` state;
5. add a narrow trace if available;
6. stop in GDB only with an explicit predicted observation;

7. inspect matching source at the earliest contradicted invariant;
8. verify all children/descriptors were cleaned despite failure.

Do not start by collecting every syscall or attaching to unrelated services. More output can erase the causal boundary.

10. Required final report

Write sections for identity, process state, fork/COW classification, exec/inheritance, wait/reparenting, signals, job-control model, injected failures, tool evidence, and three-system comparison. For every conclusion name evidence and a limit.

Example: “The child was observed as `Z` in `/proc/PID/stat` before a checked `waitpid` decoded exit 23. This proves a retained waitable status in this run; it does not measure PID-reuse windows or all procfs visibility policies.”

Conclude with cleanup: the full regression passed, every demonstration child was reaped, signals targeted only self/children, and temporary report location is known. A pass marker establishes tested invariants, not general process correctness.

Practical challenge

Purpose: produce a reproducible process/signal debugging report joining runtime, `/proc`, trace, debugger, source, and comparison evidence.

Prediction: all canonical Week 10 modes pass; expected failures are recognized; optional tools either yield scoped evidence or a clear skip.

Evidence: versions, hashes, ELF metadata, exact invocations, checked outputs, temporary logs, selected source landmarks, and explicit non-claims.

Limits: no arbitrary-process tracing, root operations, network, stress, kernel rebuild, full signal-frame capture, interactive terminal manipulation, or proof beyond the recorded versions and runs.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Debug Report

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 10-linux-processes-and-signals/challenges/day-007-debug-report.sh
```

- View lab source on GitHub